STARK-Core-v1

STARK Core Language Specification Overview

Introduction

This document provides an overview of the complete STARK core language specification. The documents in `docs/spec/` are normative for Core v1. The core language defines the general-purpose language surface (lexing, syntax, types, semantics, memory, modules, and standard library). Non-core extensions are defined separately.

Maturity: normative draft. Core v1 is the authoritative definition of the language, but it has not yet been validated by a conforming implementation. Until a reference lexer/parser/type-checker exists and every normative code example is machine-checked, readers should expect the spec to contain residual defects, and implementers should report ambiguities as spec bugs.

Design Philosophy

Core Principles

1. **Memory Safety**: Prevent common memory errors through ownership and borrowing
2. **Performance**: Zero-cost abstractions and predictable execution
3. **Clarity**: Simple, explicit syntax and semantics
4. **Pragmatism**: A minimal, implementable Core v1
5. **Interoperability**: Clear boundaries for future extensions

Language Goals

- A safe, compiled, general-purpose language core
- Compile-time guarantees for memory and type safety
- Simple, predictable semantics suitable for tooling
- Clear extension points for domain-specific features

Specification Structure

1. Lexical Grammar (01-Lexical-Grammar.md)

Defines how source code is tokenized: - **Keywords**: Control flow, declarations, types, operators - **Identifiers**: Variables, functions, types (snake_case/PascalCase) - **Literals**: Integers, floats, strings, characters, booleans - **Operators**: Arithmetic, comparison, logical, bitwise, assignment - **Comments**: Single-line (`//`) and multi-line (`/* */`) - **Whitespace**: Space, tab, newline handling

2. Syntax Grammar (02-Syntax-Grammar.md)

Defines the concrete syntax using EBNF: - **Program Structure**: Items (functions, structs, enums, traits) - **Expressions**: Precedence, associativity - **Statements**: Variable declarations, control flow, returns - **Type Syntax**: Primitives, composites, references, functions - **Pattern Matching**: Destructuring and exhaustiveness

3. Type System (03-Type-System.md)

Comprehensive type system with safety guarantees: - **Primitive Types**: Integers, floats, booleans, characters, strings - **Composite Types**: Arrays, tuples, structs, enums - **Reference Types**: Immutable and mutable references - **Ownership Model**: Move semantics, borrowing rules - **Type Inference**: Local type inference (function signatures are fully explicit) - **Trait System**: Interfaces and generic constraints

4. Semantic Analysis (04-Semantic-Analysis.md)

Rules for meaningful program validation: - **Symbol Resolution**: Scoping, shadowing, name lookup - **Type Checking**: Assignment compatibility, function calls - **Ownership Analysis**: Move tracking, borrow checking - **Control Flow**: Reachability, return path analysis - **Pattern Exhaustiveness**: Match completeness checking - **Error Reporting**: Comprehensive diagnostics with suggestions

5. Memory Model (05-Memory-Model.md)

Memory safety through compile-time analysis: - **Ownership Rules**: Single ownership, automatic cleanup - **Move Semantics**: Explicit ownership transfer - **Borrowing System**: Immutable and mutable references - **Lifetime Tracking**: Reference validity guarantees - **Stack vs Heap**: Allocation strategy and layout - **Drop System**: Automatic and manual resource cleanup

6. Standard Library (06-Standard-Library.md)

Essential types and functions for practical programming: - **Core Types**: Option, Result, Box, Vec, HashMap - **String Handling**: Unicode-aware string operations - **Collections**: Dynamic arrays, hash tables, sets - **IO Operations**: File handling, console output - **Math Functions**: Arithmetic, trigonometric, random numbers - **Error Handling**: Structured error types and propagation

7. Modules and Packages (07-Modules-and-Packages.md)

Defines module structure, visibility, and import resolution: - **Modules**: File and directory layout - **Visibility**: `pub`/`priv` rules - **Imports**: `use` paths, trees, and aliasing - **Packages**: Manifest and dependency resolution

8. Non-Core Extensions (../extensions/)

Non-core language extensions live outside Core v1 and are optional. The tensor & model type system is specified in `docs/extensions/Tensor-Model-Types.md`; the remaining AI/ML surface is sketched in `docs/extensions/AI-Extensions.md`.

Core Language Features

Variables and Mutability

```
let x = 42;             // Immutable by default
let mut y = 10;         // Explicitly mutable
const MAX_SIZE: Int32 = 1000;  // Compile-time constant
```

Functions

```
fn add(a: Int32, b: Int32) -> Int32 {
    a + b
}

fn greet(name: &str) {
    println(name);
}
```

Ownership and Borrowing

```
fn consume(s: String) {
    // s is owned here
}

fn borrow(s: &String) -> UInt64 {
    s.len()
}

fn mutate(s: &mut String) {
    s.push('!');
}
```

Implementation Phases

Phase 1: Core MVP

Goal: A complete, implementable Core v1 - Lexer and parser for core syntax - Type checker with ownership analysis - Module system and import resolution - Minimal standard library

Phase 2: Tooling and Stability

Goal: A stable core suitable for real use - Improved diagnostics and error recovery - Formatter and basic tooling support - Expanded standard library coverage

Success Criteria

Correctness

- [ ] Memory safety enforced by ownership and borrowing
- [ ] Deterministic type checking and inference
- [ ] Exhaustive match checking

Developer Experience

- [ ] Clear, actionable error messages
- [ ] Predictable module and import rules
- [ ] Stable core language surface

Next Steps

1. **Finalize Core Grammar**: Resolve remaining ambiguities in syntax and lexing
2. **Solidify Type Rules**: Confirm inference and trait constraints
3. **Define Module Rules**: Ensure deterministic resolution and visibility
4. **Validate Stdlib Surface**: Confirm minimal APIs and behaviors

This specification provides a focused foundation for implementing a safe, performant, general-purpose language core.

Conformance

A conforming Core v1 implementation MUST follow the requirements in this document. Any deviations or extensions MUST be explicitly documented by the implementation.

STARK Lexical Grammar Specification

Overview

This document defines the lexical structure of STARK - how source code is broken down into tokens.

Token Categories

1. Keywords

```
// Control Flow
if, else, match
for, while, loop, break, continue, return

// Declarations
fn, struct, enum, trait, impl, let, mut, const, type, use, mod

// Types
Int8, Int16, Int32, Int64, UInt8, UInt16, UInt32, UInt64
Float32, Float64, Bool, String, Char, Unit, str

// Visibility
pub, priv

// Module Paths and Self Type
self, Self, super, crate

// Operators
in, as

// Literals
true, false
```

2. Identifiers

```
IDENTIFIER := [a-zA-Z_][a-zA-Z0-9_]*
```

Rules: - Must start with letter or underscore - Can contain letters, digits, underscores - Case sensitive - Cannot be keywords - Maximum length: 255 characters

3. Literals

Integer Literals

```
DECIMAL_INT := [0-9] ('_'? [0-9])*
HEX_INT     := 0[xX] [0-9a-fA-F] ('_'? [0-9a-fA-F])*
BINARY_INT  := 0[bB] [01] ('_'? [01])*
OCTAL_INT   := 0[oO] [0-7] ('_'? [0-7])*

INT_SUFFIX := (i8|i16|i32|i64|u8|u16|u32|u64)

INTEGER := (DECIMAL_INT | HEX_INT | BINARY_INT | OCTAL_INT)
INT_SUFFIX?
```

Rules: - Underscores are digit separators and may appear only *between* two digits — never leading, trailing, or consecutive (`1__2` and `12_` are invalid). - A suffix fixes the literal's type: `i8`→`Int8`, `i16`→`Int16`, `i32`→`Int32`, `i64`→`Int64`, `u8`→`UInt8`, `u16`→`UInt16`, `u32`→`UInt32`, `u64`→`UInt64`. A suffixed literal whose value does not fit the named type is a compile-time error.

Examples:

```
42
1_000_000
0xFF_FF
0b1010_1010
0o755
42i32
255u8
```

Floating Point Literals

```
FLOAT_BODY := DECIMAL_INT '.' [0-9] ('_'? [0-9])* EXPONENT?
            | DECIMAL_INT EXPONENT
EXPONENT := [eE] [+-]? [0-9] ('_'? [0-9])*

FLOAT_SUFFIX := (f32|f64)

FLOAT := FLOAT_BODY FLOAT_SUFFIX?
```

Rules: - The underscore rules for integer literals apply (separators between digits only). - A suffix fixes the literal's type: `f32`→`Float32`, `f64`→`Float64`.

Examples:

```
3.14
1.0e10
2.5e-3
42.0f32
```

String Literals

```
STRING := '"' (CHAR | ESCAPE_SEQUENCE)* '"'
RAW_STRING := 'r"' .*? '"'

ESCAPE_SEQUENCE := '\' (n|t|r|0|\\|'|"|x[0-9a-fA-F]{2}|u{[0-9a-fA-
F]{1,6}})
```

Examples:

```
"Hello, World!"
"Line 1\nLine 2"
"\x41\x42\x43"
"\u{1F600}"
r"Raw string with \n literal backslashes"
```

Character Literals

```
CHAR := '\'' (CHAR_CONTENT | ESCAPE_SEQUENCE) '\''
CHAR_CONTENT := [^'\\]
```

Examples:

```
'a'
'\n'
'\x41'
'\u{1F600}'
```

Boolean Literals

```
BOOL := true | false
```

4. Operators

Arithmetic

```
+ - * / % **
```

Comparison

```
== != < <= > >=
```

Logical

```
&& || !
```

Bitwise

```
& | ^ ~ << >>
```

Assignment

```
= += -= *= /= %= **= &= |= ^= <<= >>=
```

Range

```
.. ..=
```

Other

```
? :: . -> =>
```

Notes: - `?` is the try operator (postfix). There is no ternary conditional operator; `if` is an expression. - `:` appears only as a delimiter (type annotations, struct fields), not as an operator.

5. Delimiters

```
( ) [ ] { } , ; :
```

6. Comments

```
// Single line comment
/* Multi-line comment */
/** Documentation comment */
```

Rules: - Single line comments start with `//` and continue to end of line - Multi-line comments start with `/*` and end with `*/` - Multi-line comments can be nested - Documentation comments start with `/**` and are associated with following declaration

7. Whitespace

- Space (U+0020)
- Tab (U+0009)
- Newline (U+000A)
- Carriage Return (U+000D)

Whitespace separates tokens and is otherwise ignored. Statement termination is defined by semicolons in the syntax grammar.

8. Token Precedence

When multiple token patterns could match: 1. Keywords take precedence over identifiers 2. Longer operators take precedence over shorter ones 3. Comments are ignored in token stream

9. Reserved Tokens

Reserved for future use (recognized as keywords but not used by any Core v1 grammar production):

```
async, await, yield, where, macro, unsafe, extern, import, export,
null,
and, or, not, is, dyn
```

Lexical Analysis Rules

1. **Maximal Munch**: Always match the longest possible token
2. **Whitespace**: Ignored except for token separation
3. **Encoding**: Source files must be valid UTF-8

Error Handling

Invalid tokens should produce specific error messages: - Invalid character: “Unexpected character ‘X’ at line Y:Z” - Unterminated string: “Unterminated string literal at line Y:Z” - Invalid number: “Invalid number format at line Y:Z” ## Conformance A conforming Core v1 implementation MUST follow the requirements in this document. Any deviations or extensions MUST be explicitly documented by the implementation.

STARK Syntax Grammar Specification

Overview

This document defines the concrete syntax grammar for STARK using Extended Backus-Naur Form (EBNF).

Grammar Notation

```
::= means "is defined as"
|   means "or" (alternative)
?   means "optional" (zero or one)
*   means "zero or more"
+   means "one or more"
()  means "grouping"
[]  means "optional"
{}  means "zero or more repetitions"
```

Top-Level Grammar

Program

```
Program ::= Item*

Item ::= Visibility? (Function
       | Struct
       | Enum
       | Trait
       | Impl
       | Const
       | TypeAlias
       | Use
       | Module)

Visibility ::= 'pub' | 'priv'
```

Generic Parameters and Arguments

```
GenericParams ::= '<' GenericParam (',' GenericParam)* ','? '>'

GenericParam ::= IDENTIFIER (':' TraitBounds)?

TraitBounds ::= TraitBound ('+' TraitBound)*

TraitBound ::= Path GenericArgs?

GenericArgs ::= '<' GenericArg (',' GenericArg)* ','? '>'

GenericArg ::= Type
             | IDENTIFIER '=' Type          // Associated type
binding, e.g. Iterator<Item = T>
```

Notes: - Generic parameters may appear on functions, structs, enums, traits, impl blocks, and type aliases. - Generic arguments in *expressions* are inferred at the use site. The only explicit form is `path::<Args>` on a path expression (see `PathExpression`), for calls where inference is impossible, e.g. `size_of::<Int32>()`.

Function Definition

```
Function ::= FunctionSig Block

FunctionSig ::= 'fn' IDENTIFIER GenericParams? '(' ParameterList?
')' ('->' ReturnType)?

ReturnType ::= Type | '!'                   // '!' is the never
type (diverging function)

ParameterList ::= Receiver (',' Parameter)* ','?
```

```
               | Parameter (',' Parameter)* ','?

Receiver ::= 'self'
           | '&' 'self'
           | '&' 'mut' 'self'

Parameter ::= IDENTIFIER ':' Type
            | 'mut' IDENTIFIER ':' Type

Block ::= '{' Statement* Expression? '}'
```

Block semantics: - The optional final `Expression` has no terminating semicolon and is the block's value; a block without one evaluates to `Unit`. - Disambiguation: at each position inside a block, the parser first attempts to parse a `Statement`; a trailing `Expression` is recognized only immediately before `}`. An expression followed by `;` is always a statement.

Notes: - A receiver is only valid on functions declared inside a `trait` or `impl` block. - A function without a `->` annotation returns `Unit`. Return types are never inferred.

Data Type Definitions

```
Struct ::= 'struct' IDENTIFIER GenericParams? '{' FieldList? '}'

FieldList ::= Field (',' Field)* ','?

Field ::= IDENTIFIER ':' Type
        | 'pub' IDENTIFIER ':' Type

Enum ::= 'enum' IDENTIFIER GenericParams? '{' VariantList? '}'

VariantList ::= Variant (',' Variant)* ','?

Variant ::= IDENTIFIER
          | IDENTIFIER '(' TypeList ')'
          | IDENTIFIER '{' FieldList '}'

Trait ::= 'trait' IDENTIFIER GenericParams? '{' TraitItem* '}'

TraitItem ::= FunctionSig ';'               // Required method
(signature only)
            | FunctionSig Block             // Method with default
body
            | AssociatedType

AssociatedType ::= 'type' IDENTIFIER ';'

Impl ::= 'impl' GenericParams? Type '{' ImplItem* '}'
       | 'impl' GenericParams? TraitBound 'for' Type '{' ImplItem*
'}'

ImplItem ::= Visibility? Function
```

```
           | 'type' IDENTIFIER '=' Type ';' // Associated type
value
```

Module Definition

```
Module ::= 'mod' IDENTIFIER ';'
        | 'mod' IDENTIFIER ModuleBlock

ModuleBlock ::= '{' Item* '}'
```

Type Alias

```
TypeAlias ::= 'type' IDENTIFIER GenericParams? '=' Type ';'
```

Statements

```
Statement ::= ';'                     // Empty statement
            | Expression ';'          // Expression statement
            | BlockExpression ';'?    // Block-formed expression
statement
            | 'let' LetStatement
            | 'return' Expression? ';'
            | 'break' Expression? ';'
            | 'continue' ';'

BlockExpression ::= Block
                  | IfExpression
                  | MatchExpression
                  | LoopExpression

LetStatement ::= IDENTIFIER ':' Type ';'
               | 'mut' IDENTIFIER ':' Type ';'
               | IDENTIFIER ':' Type '=' Expression ';'
               | IDENTIFIER '=' Expression ';'
               | 'mut' IDENTIFIER ':' Type '=' Expression ';'
               | 'mut' IDENTIFIER '=' Expression ';'
```

Block-formed expression statements: - An expression whose outermost form is a block, `if`, `match`, `loop`, `while`, or `for` may stand as a statement without a terminating semicolon (`if ready { start(); }` mid-block is a statement). - Statement parsing is greedy: when a block-formed expression begins at statement position and is not followed by `;`, it is complete as a statement — a following token does NOT extend it into a larger expression (`{ if c { } - 1; }` is an `if` statement followed by an error, not a subtraction). Parenthesize to use the value: `(if c { 1 } else { 2 }) - 1`. - Exception (trailing expression): a block-formed expression immediately before the closing `}` of its block and not followed by `;` is the block's trailing `Expression` (its value), not a statement — so `fn max(...) -> T { if a > b { a } else { b } }` returns the `if`'s value.

Expressions

```
Expression ::= AssignmentExpression

AssignmentExpression ::= RangeExpression
                       | RangeExpression AssignOp
AssignmentExpression

AssignOp ::= '=' | '+=' | '-=' | '*=' | '/=' | '%=' | '**='
           | '&=' | '|=' | '^=' | '<<=' | '>>='
```

Place Expressions

The left-hand side of an assignment must be a *place expression* — an expression that denotes a memory location. Place expressions are:

```
PlaceExpression ::= Path                              // Variable
                  | PlaceExpression '.' IDENTIFIER    // Field
                  | PlaceExpression '.' INTEGER       // Tuple
field
                  | PlaceExpression '[' Expression ']' // Index
                  | '*' Expression                    //
Dereference
                  | '(' PlaceExpression ')'
```

Assignment to any other expression form is a compile-time error (checked semantically; the grammar accepts any expression on the left).

```
RangeExpression ::= LogicalOrExpression (('..' | '..=')
LogicalOrExpression)?

LogicalOrExpression ::= LogicalAndExpression ('||'
LogicalAndExpression)*

LogicalAndExpression ::= EqualityExpression ('&&'
EqualityExpression)*

EqualityExpression ::= RelationalExpression (('==' | '!=')
RelationalExpression)?

RelationalExpression ::= BitwiseOrExpression (('<' | '<=' | '>' |
'>=') BitwiseOrExpression)?

BitwiseOrExpression ::= BitwiseXorExpression ('|'
BitwiseXorExpression)*

BitwiseXorExpression ::= BitwiseAndExpression ('^'
BitwiseAndExpression)*

BitwiseAndExpression ::= ShiftExpression ('&' ShiftExpression)*

ShiftExpression ::= AdditiveExpression (('<<' | '>>')
AdditiveExpression)*
```

```
AdditiveExpression ::= MultiplicativeExpression (('+' | '-')
MultiplicativeExpression)*

MultiplicativeExpression ::= ExponentiationExpression (('*' | '/'
| '%') ExponentiationExpression)*

ExponentiationExpression ::= CastExpression ('**'
ExponentiationExpression)?

CastExpression ::= UnaryExpression ('as' Type)*

UnaryExpression ::= PostfixExpression
                  | '-' UnaryExpression
                  | '!' UnaryExpression
                  | '~' UnaryExpression
                  | '&' 'mut'? UnaryExpression // Reference
(immutable or mutable borrow)
                  | '*' UnaryExpression        // Dereference

PostfixExpression ::= PrimaryExpression
                    | PostfixExpression '(' ArgumentList?
')'     // Function call
                    | PostfixExpression '[' Expression
']'        // Index (or slice, when the index is a range)
                    | PostfixExpression '.'
IDENTIFIER            // Field access / method reference
                    | PostfixExpression '.'
INTEGER               // Tuple access
                    | PostfixExpression
'?'                       // Try operator

ArgumentList ::= Expression (',' Expression)* ','?

PrimaryExpression ::= PathExpression
                    | Literal
                    | '(' Expression ')'    // Grouping
                    | TupleLiteral
                    | ArrayLiteral
                    | StructLiteral
                    | IfExpression
                    | MatchExpression
                    | LoopExpression
                    | Block

PathExpression ::= Path ('::' GenericArgs)?  // e.g. x,
String::from, Color::Red, size_of::<Int32>
```

Notes: - `path::<Args>` (explicit generic arguments on a path expression) is permitted only when type inference cannot determine the arguments, e.g. `size_of::<Int32>()`. It is the only form of expression-level generic arguments in

Core v1. - Chained comparisons (`a < b < c`, `a == b == c`) are syntax errors: the relational and equality productions are non-associative (at most one operator). Parenthesize to compare a Bool result explicitly.

Control Flow Expressions

```
IfExpression ::= 'if' Expression Block ('else' 'if' Expression
Block)* ('else' Block)?

MatchExpression ::= 'match' Expression '{' MatchArm* '}'

MatchArm ::= Pattern '=>' Expression ','?

Pattern ::= Literal
          | '_'                                  // Wildcard
          | IDENTIFIER                           // Binding (or
unit variant/const, see note)
          | Path                                 // Unit enum
variant or const, e.g. Color::Red
          | Path '(' PatternList? ')'            // Tuple enum
variant, e.g. Option::Some(x)
          | Path '{' FieldPatternList? '}'       // Struct or
struct enum variant
          | '(' PatternList? ')'                 // Tuple
          | '[' PatternList? ']'                 // Array

PatternList ::= Pattern (',' Pattern)* ','?

FieldPatternList ::= FieldPattern (',' FieldPattern)* ','?

FieldPattern ::= IDENTIFIER ':' Pattern
               | IDENTIFIER                      // Shorthand:
binds field to same-named variable

LoopExpression ::= 'loop' Block
                 | 'while' Expression Block
                 | 'for' IDENTIFIER 'in' Expression Block
```

Note on pattern name resolution: a single `IDENTIFIER` pattern that resolves to a unit enum variant or a constant in scope matches by value; otherwise it introduces a new binding. Multi-segment `Path` patterns always match by value.

Literals

```
Literal ::= INTEGER
          | FLOAT
          | STRING
          | CHAR
          | BOOLEAN

TupleLiteral ::= '(' ')'                                  // Unit
value
```

```
               | '(' Expression ',' ')'                   //
Single-element tuple
               | '(' Expression (',' Expression)+ ','? ')' // N-
element tuple

ArrayLiteral ::= '[' ExpressionList? ']'
               | '[' Expression ';' Expression ']'        //
Repeat: [value; count]

ExpressionList ::= Expression (',' Expression)* ','?

StructLiteral ::= Path '{' FieldInitList? '}'

FieldInitList ::= FieldInit (',' FieldInit)* ','?

FieldInit ::= IDENTIFIER ':' Expression
            | IDENTIFIER                    // Shorthand: field:
field
```

Notes: - `(expr)` is grouping; `(expr,)` is a single-element tuple; `()` is the unit value. The trailing comma distinguishes the 1-tuple from grouping. - In `[value; count]`, `count` must be a compile-time constant expression of an unsigned integer type, and `value`'s type must implement `Copy` (the value is copied `count` times).

Struct Literal Restriction

To avoid ambiguity with block-taking constructs, a struct literal may NOT appear as the outermost expression in: - the condition of an `if` or `while`, - the scrutinee of a `match`, - the iterable of a `for`.

Wrap the struct literal in parentheses in these positions:

```
if (Config { verbose: true }).verbose { do_thing(); }
```

Types

```
Type ::= PrimitiveType
       | Path GenericArgs?                 // Named type, possibly
generic: Vec<Int32>, Option<T>
       | '[' Type ';' INTEGER ']'          // Array type
       | '[' Type ']'                      // Slice type (unsized;
used behind references)
       | TupleType
       | '(' Type ')'                      // Grouping (the type
T, not a tuple)
       | '&' Type                          // Reference type
       | '&' 'mut' Type                    // Mutable reference
type
       | 'fn' '(' TypeList? ')' ('->' Type)?  // Function type
(non-capturing)

TupleType ::= '(' ')'                      // Unit type
            | '(' Type ',' ')'             // Single-element tuple
```

```
type
            | '(' Type (',' Type)+ ','? ')' // N-element tuple
type

PrimitiveType ::= 'Int8' | 'Int16' | 'Int32' | 'Int64'
                | 'UInt8' | 'UInt16' | 'UInt32' | 'UInt64'
                | 'Float32' | 'Float64'
                | 'Bool' | 'Char' | 'String' | 'Unit' | 'str'

TypeList ::= Type (',' Type)* ','?
```

Notes: - A function type without `->` returns `Unit`. - The never type `!` may appear only as a function return type (see `ReturnType`). - `(T)` is the type `T` (grouping); the single-element tuple type is `(T,)`, mirroring `TupleLiteral` on the value side.

Other Constructs

```
Const ::= 'const' IDENTIFIER ':' Type '=' Expression ';'

Use ::= 'use' UseTree ';'

UseTree ::= Path ('as' IDENTIFIER)?
          | Path '::' '*'
          | Path '::' 'self'
          | Path '::' '{' UseTreeList? '}'

UseTreeList ::= UseTree (',' UseTree)* ','?

Path ::= PathSegment ('::' PathSegment)*
PathSegment ::= IDENTIFIER | PrimitiveType | 'self' | 'Self' |
'super' | 'crate'
```

Notes: - `Self` is valid only inside a `trait` or `impl` block, and only as the *first* segment of a path. As a complete one-segment path in type position it denotes the implementing type; with further segments it projects an associated item, e.g. `Self::Item`. - `self`, `super`, and `crate` are likewise valid only as leading segments (`self`/`super` may repeat at the front: `super::super::x`). - A `PrimitiveType` keyword is valid only as the *first* segment, where it names the primitive's associated items: `String::from(s)`. (Primitive type names are keywords in the lexical grammar, so without this rule such calls would be unparseable.)

Operator Precedence (Highest to Lowest)

1. Primary expressions, field access, array access, function calls, try operator (`?`)
2. Unary operators (-, !, ~, &, &mut, *)
3. Cast (`as`)
4. Exponentiation (**)
5. Multiplicative (*, /, %)
6. Additive (+, -)
7. Shift (<<, >>)
8. Bitwise AND (&)

9. Bitwise XOR (^)
10. Bitwise OR (|)
11. Relational (<, <=, >, >=)
12. Equality (==, !=)
13. Logical AND (&&)
14. Logical OR (||)
15. Range (.., ..=)
16. Assignment (=, +=, -=, etc.)

Associativity

- Left associative: Most binary operators
- Right associative: Assignment operators, exponentiation
- Non-associative: Comparison operators, range operators

Statement vs Expression

- Statements do not return values and end with semicolons (block-formed expression statements may omit the semicolon — see Statements)
- Expressions return values and can be used as statements with semicolons
- Blocks are expressions (return value of last expression, or Unit if none)
- Control flow constructs are expressions

Whitespace and Semicolons

- Semicolons are required to terminate statements, except after block-formed expression statements (see Statements)
- Semicolons are optional after the last expression in a block
- Newlines are not significant except for line comments
- Trailing commas are allowed in lists

Parsing Notes

- `>>` in a generic argument position (e.g. `Vec<Vec<Int32>>`) MUST be re-tokenized as two `>` tokens by the parser.
- In expression position, a bare `<` after an identifier is always the relational operator; explicit generic arguments require the `::<` form (turbofish), so no lookahead disambiguation is required.

Grammar Extensions for Future Features

Reserved grammar constructs for later implementation:

```
// Async functions (future)
AsyncFunction ::= 'async' 'fn' IDENTIFIER '(' ParameterList? ')'
('->' Type)? Block
```

```
// Lambda expressions / capturing closures (future)
Lambda ::= '|' ParameterList? '|' (Expression | Block)

// Open-ended ranges (future)
OpenRange ::= Expression '..' | '..' Expression | '..'

// Lifetime annotations (future)
LifetimeParam ::= '\'' IDENTIFIER
```

Conformance

A conforming Core v1 implementation MUST follow the requirements in this document. Any deviations or extensions MUST be explicitly documented by the implementation.

STARK Type System Specification

Overview

STARK features a static type system with type inference, ownership semantics, and memory safety guarantees.

Core Principles

1. **Static Typing**: All types resolved at compile time
2. **Type Inference**: Types can be inferred when unambiguous
3. **Memory Safety**: No null pointer dereferences, no use-after-free
4. **Ownership**: Clear ownership and borrowing rules
5. **Zero-cost Abstractions**: Type safety without runtime overhead

Primitive Types

Integer Types

```
Int8    // 8-bit signed integer (-128 to 127)
Int16   // 16-bit signed integer (-32,768 to 32,767)
Int32   // 32-bit signed integer (-2^31 to 2^31-1)
Int64   // 64-bit signed integer (-2^63 to 2^63-1)

UInt8   // 8-bit unsigned integer (0 to 255)
UInt16  // 16-bit unsigned integer (0 to 65,535)
UInt32  // 32-bit unsigned integer (0 to 2^32-1)
UInt64  // 64-bit unsigned integer (0 to 2^64-1)
```

Default integer type is `Int32` for literals that fit, `Int64` otherwise.

Floating Point Types

```
Float32  // 32-bit IEEE 754 floating point
Float64  // 64-bit IEEE 754 floating point
```

Default floating point type is `Float64`.

Other Primitive Types

```
Bool    // Boolean: true or false
Char    // Unicode scalar value (32-bit)
Unit    // Unit type: () - represents no meaningful value
str     // String slice (unsized), typically used behind
references: &str
```

String Type

```
String  // UTF-8 encoded string, heap-allocated, growable
```

String Slice Type

```
// str is an unsized string slice type. It is used via references.
let s: &str = "hello";
let owned: String = String::from(s);
```

Never Type

`!` (the never type) is the type of expressions that never produce a value, such as calls to `panic`. It may appear only as a function return type.

```
fn panic(message: &str) -> !
```

Rules: - An expression of type `!` coerces to any other type. This allows, for example, `panic(...)` to appear in one arm of an `if` or `match` whose other arms produce a value. - A function returning `!` MUST NOT return normally.

Composite Types

Array Types

```
[T; N]   // Fixed-size array of N elements of type T (sized)
[T]      // Slice of elements of type T (unsized)
```

A slice `[T]` is an *unsized* view into a contiguous sequence (an array, or the elements of a `Vec<T>`). Like `str`, it is used behind references:

```
let fixed: [Int32; 5] = [1, 2, 3, 4, 5];
let view: &[Int32] = &fixed;          // Array reference coerces
to slice reference
let part: &[Int32] = &fixed[1..4];    // Slicing with a range
yields &[T]
```

A local variable cannot have the bare unsized type `[T]`; use `[T; N]` for a fixed-size array or `Vec<T>` (standard library) for a growable one.

Indexing rules: - `expr[i]` where `i` is an integer denotes a *place* of type `T` (bounds-checked; traps on out-of-bounds). - `expr[r]` where `r` is a `Range` denotes a *place* of the unsized slice type `[T]`; traps if the range is out of bounds or inverted. Because `[T]` is unsized, a slice place must be borrowed immediately: `&expr[r]` has type `&[T]` and `&mut expr[r]` has type `&mut [T]`.

Range Type

The range operators produce values of the standard library `Range<T>` type:

```
let r = 0..10;      // Range<Int32>: 0,1,...,9 (half-open)
let ri = 0..=9;     // RangeInclusive<Int32>: 0,1,...,9
```

Ranges over integer types implement `Iterator` and may be used with `for` loops and slicing.

Tuple Types

```
()           // Empty tuple (Unit type)
(T,)         // Single-element tuple
(T1, T2)     // Two-element tuple
(T1, T2, T3) // Three-element tuple
// ... up to reasonable limit (e.g., 16 elements)
```

Examples:

```
let empty: () = ();
let single: (Int32,) = (42,);
let pair: (Int32, String) = (42, "hello");
```

Struct Types

```
struct Point {
    x: Float64,
    y: Float64
}

struct Person {
    name: String,
    age: Int32,
    pub email: String  // Public field
}
```

Restriction (Core v1): struct and enum declarations MUST NOT declare fields whose *written* type is a reference type (`&T`, `&mut T`). Instantiating a generic type with a reference type argument (e.g. `Option<&T>`) is permitted; the resulting value is *borrow-carrying* — see "References and Lifetimes" below.

Enum Types

```
enum Color {
    Red,
    Green,
    Blue
}

enum Option<T> {
    Some(T),
    None
}

enum Result<T, E> {
    Ok(T),
    Err(E)
}
```

Reference Types

```
&T       // Immutable reference to T
&mut T   // Mutable reference to T
```

Function Types

Function types are written with the `fn` keyword and denote *non-capturing* functions (named functions or function references). Capturing closures are a future extension.

```
fn(T1, T2) -> R    // Function taking T1, T2 and returning R
fn() -> R          // Function taking no parameters, returning R
fn(T)              // Function taking T, returning Unit
```

Type Aliases

```
type Age = Int32;
type Point2D = (Float64, Float64);
type ErrorCode = Int32;
```

The Self Type

Within a `trait` or `impl` block, `Self` refers to the implementing type:

```
trait Eq {
    fn eq(&self, other: &Self) -> Bool;
}
```

Ownership and Borrowing

Ownership Rules

1. Each value has exactly one owner
2. When the owner goes out of scope, the value is dropped
3. Values can be moved (ownership transfer) or borrowed (temporary access)

Move Semantics

```
let a = String::from("hello");
let b = a;  // a is moved to b, a is no longer valid
// print(a);  // Error: use of moved value
```

Borrowing Rules

1. You can have either one mutable reference or any number of immutable references
2. References must always be valid (no dangling pointers)
3. References cannot outlive the data they refer to

```
fn borrow_immutable(s: &String) {
    // Can read but not modify
}

fn borrow_mutable(s: &mut String) {
    // Can read and modify
}

let mut text = String::from("hello");
borrow_immutable(&text);       // OK
borrow_mutable(&mut text);     // OK
```

References and Lifetimes (Core v1)

Core v1 has no lifetime annotations. Instead, it enforces the following conservative rules, which are normative:

1. **No declared reference fields.** Struct, enum variant, and tuple struct declarations MUST NOT write a reference type (`&T`, `&mut T`) as a field type. (Generic *instantiation* with a reference argument is allowed; see Borrow-Carrying Types below.)
2. **References derive from parameters.** A function may return a reference (or borrow-carrying value) only if every control-flow path derives it from one of its reference parameters (directly, by field/index projection, or by calling a function

that itself obeys this rule). Returning a reference derived from a local variable or a by-value parameter is a compile-time error.
3. **Shortest-input-lifetime rule.** The lifetime of a returned reference (or borrow-carrying value) is the *shortest* of the lifetimes of all reference parameters from which it could have been derived. Callers MUST treat it as invalidated as soon as the shortest-lived of those arguments is invalidated.
4. **Local borrows are lexically scoped.** A borrow bound to a variable (`let r = &x;`) lasts from its creation to the end of its enclosing block (or until the borrower is explicitly dropped with `drop(...)`). The borrow checker enforces the exclusive/shared rules over that entire region — even if the reference is never used again. To end such a borrow early, introduce an inner block or call `drop`. A *temporary* borrow that is not bound to a variable (e.g. `f(&x);`) ends at the end of its enclosing statement, so `f(&x); g(&mut x);` is legal.

```
let mut s = String::from("hello");
{
    let r = &s;           // Borrow begins
    println(r.as_str());
}                         // Borrow ends with the block
s.push('!');              // OK: no live borrow
```

Example of rule 3:

```
fn longest(x: &str, y: &str) -> &str {
    if x.len() > y.len() { x } else { y }
}
// The returned reference is valid only while BOTH x's and y's
referents
// are valid, regardless of which branch was taken.
```

These rules reject some safe programs (that is intentional). Lifetime annotations that relax rule 3, user structs with declared reference fields, and non-lexical (last-use) borrow regions are future extensions; because the lexical rule is strictly more conservative, adopting last-use regions later cannot break conforming programs.

Borrow-Carrying Types

A type is **borrow-carrying** if it is: - a reference type `&T` or `&mut T`; or - a generic type with at least one borrow-carrying type argument (e.g. `Option<&T>`, `(Int32, &str)`); or - an implementation-provided borrowed view type (the standard library's borrowed iterators such as `VecIter<T>`, `CharsIter`, `KeysIter<K>`, and the slice types behind references).

A borrow-carrying value is treated *as if it were a reference* for all rules in this section: - It counts as a live borrow of the value(s) it was derived from — shared for &-derived values, exclusive for `&mut`-derived values — for the borrow region defined by rule 4. - It MUST NOT be stored in a user-declared struct or enum field, returned except under rules 2–3, or otherwise escape the lifetime of its source. - Binding it with `let` is permitted; the binding is subject to rule 4.

This is a taint rule: borrow-carrying-ness propagates through generic instantiation and function returns, and the checker tracks the source variables each borrow-carrying value may borrow from. It is what makes APIs like

`Vec::get(&self, i) -> Option<&T>` sound without lifetime annotations: the returned `Option<&T>` is an active shared borrow of the `Vec` until it goes out of scope.

Type Inference

Local Type Inference

Types of `let` bindings are inferred from their initializers. Inference is local: it uses the initializer expression and, where necessary, later uses within the same function body. Function signatures are always fully explicit, so inference never crosses function boundaries.

```
let x = 42;             // Inferred as Int32
let y = 3.14;           // Inferred as Float64
let z = [1, 2, 3];      // Inferred as [Int32; 3]
```

Function Return Types

Function return types are NOT inferred. A function without a `->` annotation returns `Unit`.

```
fn add(a: Int32, b: Int32) -> Int32 {
    a + b
}

fn greet(name: &str) {      // Returns Unit
    println(name);
}
```

Generic Type Inference

Generic arguments at call sites are inferred from argument and expected types:

```
fn identity<T>(x: T) -> T {
    x
}

let result = identity(42);  // T inferred as Int32
```

Subtyping and Coercion

Numeric Coercions

No implicit numeric conversions. Explicit casting required:

```
let x: Int32 = 42;
let y: Int64 = x as Int64;  // Explicit cast required
```

Cast semantics (`as`): - Between integer types: value-preserving if in range; a cast whose value does not fit the target type is a runtime error and MUST trap. - Between floating point types: rounds to nearest representable value. - Integer to float and float to integer: float-to-int truncates toward zero and MUST trap if the result does not fit the target type (including NaN/Inf).

Reference Coercions

```
&mut T -> &T        // Mutable reference to immutable reference
&T -> &T            // Same type (identity)
```

Array to Slice Coercion

```
&[T; N] -> &[T]     // Array reference to slice reference
&mut [T; N] -> &mut [T]  // Mutable array reference to mutable
slice reference
```

Never Coercion

```
! -> T              // The never type coerces to any type
```

Trait System (Basic)

Trait Definition

```
trait Display {
    fn fmt(&self) -> String;
}

trait Eq {
    fn eq(&self, other: &Self) -> Bool;
}
```

Trait Implementation

```
impl Display for Int32 {
    fn fmt(&self) -> String {
        // Convert integer to string
    }
}

impl Eq for Point {
    fn eq(&self, other: &Point) -> Bool {
        self.x == other.x && self.y == other.y
    }
}
```

Associated Types

Traits may declare associated types; implementations assign them:

```
trait Iterator {
    type Item;
    fn next(&mut self) -> Option<Self::Item>;
}

struct Counter { count: Int32 }

impl Iterator for Counter {
    type Item = Int32;
    fn next(&mut self) -> Option<Int32> {
        self.count += 1;
        Some(self.count)
    }
}
```

Bounds may constrain associated types with bindings: `I: Iterator<Item = T>`.

Type Checking Rules

Assignment Compatibility

```
let x: T = expr;  // expr must have type T or be coercible to T
```

Function Call Compatibility

```
fn f(param: T) { ... }
f(arg);  // arg must have type T or be coercible to T
```

Arithmetic Operations

```
// Binary arithmetic requires same types
let result = x + y;  // x and y must have the same numeric type

// Comparison operators
let cmp = x < y;     // x and y must have the same comparable type
```

Logical Operations

```
let both = a && b;    // a and b must be Bool
let negated = !x;     // x must be Bool
```

For Loops

`for x in expr` requires `expr` to have a type that implements `Iterator`; the loop variable binds successive `Item` values. Integer ranges implement `Iterator`; slices and collections provide `.iter()` methods (see the standard library specification).

```
for i in 0..10 {
    println(i.fmt());
}
```

Control Flow Typing

- **`if`/`else`**: an `if` with an `else` is an expression whose branches must unify to a single type (identical types, or unifiable via the `!` coercion — e.g. one branch may `panic`). An `if` *without* `else` has type `Unit`, and its branch must also have type `Unit`.
- **`match`**: all arm expressions must unify to a single type (with `!` coercion permitted per arm).
- **`loop`**: a `loop` expression's type is the unified type of the operands of its `break value;` statements; if every `break` is bare (no value) or the loop has no `break`, the type is `Unit` (a `loop` with no `break` has type `!`).
- **`while` / `for`**: always have type `Unit`. `break` inside `while`/`for` MUST NOT carry a value.
- **Block**: the type of the trailing expression, or `Unit` if there is none.

Method Calls and Auto-Borrowing

A method call `recv.m(args)` is resolved as follows:

1. **Candidate collection.** Candidates are, in priority order:
    1. inherent methods — `fn m` in `impl RecvType { ... }` blocks;
    2. trait methods — `fn m` from any trait that RecvType implements, where the trait is *in scope* (defined in, or imported via `use` into, the current module, or in the prelude). Inherent methods shadow trait methods of the same name.
2. **Ambiguity.** If two or more traits in scope supply an applicable `m` and no inherent method exists, the call is a compile-time error. Disambiguate with a fully-qualified call, passing the receiver explicitly: `TraitName::m(&recv, args)`.
3. **Receiver coercion (auto-borrowing).** Let `S` be the receiver expression's type. Candidates are matched trying these receiver forms in order: `self: S` (by value), `self: &S` (auto-borrow: the compiler inserts `&`), `self: &mut S` (auto-mutable-borrow: the compiler inserts `&mut`; requires the receiver to be a mutable place). Auto-borrows follow the normal borrow rules.
4. **Auto-dereference.** If `S` is `&T` or `&mut T` and no candidate matches on `S`, resolution retries with `T` (one level of dereference, applied repeatedly for nested references). Combined with step 3 this allows `(&v).len()` and `v.len()` to resolve identically.
5. **Visibility.** Private methods are callable only per the module rules (07-Modules-and-Packages.md).

Trait methods can always be called in fully-qualified function form — `Display::fmt(&x)` — since a method is an ordinary function whose first parameter is the receiver.

Operators and Traits

Operator expressions on **primitive types** have built-in meaning (Numeric Semantics below). On **generic type parameters**, operators desugar to trait method calls, so the corresponding bound is required:

Operator	Requires bound	Desugars to
`==`, `!=`	`T: Eq`	`Eq::eq(&a, &b)` (negated for `!=`)
`<`, `<=`, `>`, `>=`	`T: Ord`	`Ord::cmp(&a, &b)` compared to `Ordering`
`+ - * / % **`, bitwise, shifts	`T: Num`	primitive operation after monomorphization

`Num` is a compiler-known marker trait implemented by exactly the built-in numeric types (`Int8`–`Int64`, `UInt8`–`UInt64`, `Float32`, `Float64`); user types cannot implement it in Core v1. Operator overloading for user-defined types (Add/Sub/… traits) is a future extension. `&&`, `||`, and `!` (on `Bool`) are built-in, short-circuiting, and not overloadable.

```
fn max<T: Ord>(a: T, b: T) -> T {
    if a > b { a } else { b }   // a > b desugars to Ord::cmp(&a,
&b)
}
```

Copy and Drop (Soundness Rules)

- `Copy` may be implemented for a type only if **all** of its fields are `Copy`. Violations are compile-time errors.
- A type MUST NOT implement both `Copy` and `Drop` (a copyable type would run its destructor once per copy).
- `Drop::drop` MUST NOT be called explicitly; use the free function `drop(value)`, which takes ownership and runs the destructor exactly once. After `drop(v)`, `v` is moved-from.
- Every owned value's destructor runs **exactly once**: at end of scope, at explicit `drop`, or when its owner is consumed — never twice. For values moved on only some control-flow paths, the implementation MUST track initialization state (e.g. drop flags) to preserve exactly-once semantics.
- **Partial moves**: moving a field out of a struct is permitted only if the struct's type does not implement `Drop`. After a partial move, the whole value may no longer be used or moved, but its remaining fields may be.
- **Reinitialization**: assigning a new value to a moved-from `let mut` variable makes it valid again (definite-assignment tracking).
- Moving an element out of an indexed place (`v[i]`) is a compile-time error; use APIs that transfer ownership explicitly (e.g. `Vec::remove`, `Vec::pop`).

Error Types and Handling

Option Type

```
enum Option<T> {
    Some(T),
    None
}
```

Result Type

```
enum Result<T, E> {
    Ok(T),
    Err(E)
}
```

Error Propagation

```
fn might_fail() -> Result<Int32, String> {
    // ...
}

fn caller() -> Result<Int32, String> {
    let value = might_fail()?;  // Early return on error
    Ok(value * 2)
}
```

Try Operator (?) Typing

The try operator is defined for `Result<T, E>` and `Option<T>`: - If `expr` has type `Result<T, E>`, then `expr?` has type `T` and propagates `Err(E)` to the nearest enclosing function returning `Result<_, E>`. - If `expr` has type `Option<T>`, then `expr?` has type `T` and propagates `None` to the nearest enclosing function returning `Option<_>`.

The enclosing function's return type must be compatible with the propagated type.

Generics (Core v1)

Core v1 supports parametric polymorphism for functions, structs, enums, and traits.

Rules: - Generic parameters are introduced with `<T, U, ...>` after the item name. - Generic parameters are in scope within the item body and signatures. - All generic parameters used in an item MUST be declared by that item. - Instantiation occurs at use sites; Core v1 permits monomorphization or dictionary-passing, but the observable behavior MUST be equivalent. - Generic arguments in expressions are inferred. When inference is impossible (no parameter or usable result type mentions

the type parameter), explicit arguments are supplied with the turbofish form on a path expression: `size_of::<Int32>()`. This is the only expression-level generic-argument syntax in Core v1.

```
struct Pair<T> {
    first: T,
    second: T
}

fn max<T: Ord>(a: T, b: T) -> T {
    if a > b { a } else { b }
}
```

Trait Bounds

```
fn max<T: Ord>(a: T, b: T) -> T { if a > b { a } else { b } }
```

Rules: - A bound `T: Trait` requires a visible `impl Trait for T`. - Multiple bounds are allowed: `T: TraitA + TraitB`. - Bounds may bind associated types: `I: Iterator<Item = T>`.

Trait Coherence (Core v1)

To avoid ambiguous implementations, Core v1 applies the orphan rule: - An `impl` is valid only if either the trait or the type is defined in the current package.

Additionally: - If multiple applicable `impl` blocks could apply to the same type at a call site, the program is ill-formed. - Blanket implementations (e.g., `impl<T: Trait> OtherTrait for T`) are permitted but must not violate coherence.

Numeric Semantics (Core v1)

- Integer overflow and underflow are runtime errors and MUST trap. This applies in every build configuration; there is no mode in which overflow wraps silently.
- Division or modulo by zero is a runtime error and MUST trap.
- Floating-point operations follow IEEE-754 semantics (NaN, +/-Inf).

Type Safety Guarantees

1. **No null pointer dereferences**: Option type prevents null access
2. **No buffer overflows**: Array bounds checking
3. **No use-after-free**: Ownership system prevents dangling pointers
4. **No data races**: Borrowing rules prevent concurrent access violations
5. **No memory leaks**: Automatic memory management through ownership

Type System Extensions (Future)

Lifetime Parameters

```
fn longest<'a>(x: &'a str, y: &'a str) -> &'a str {
    if x.len() > y.len() { x } else { y }
}
```

Trait Objects

Dynamic dispatch via `dyn Trait` is a future extension; `dyn` is a reserved keyword.

Capturing Closures

Lambda expressions that capture their environment are a future extension; the `fn(...)` function types in Core v1 are non-capturing.

Implementation Notes

Type Representation

- Primitive types: Direct machine representation
- Composite types: Laid out according to platform ABI
- References: Pointers with compile-time tracking
- Enums: Tagged unions with optimal layout

Type Checking Algorithm

1. Parse source into AST
2. Build symbol table with declarations
3. Perform local type inference by unification within function bodies (function signatures are fully annotated, so no global inference is needed)
4. Check type constraints and ownership rules
5. Generate type-annotated AST for code generation ## Conformance A conforming Core v1 implementation MUST follow the requirements in this document. Any deviations or extensions MUST be explicitly documented by the implementation.

STARK Semantic Analysis Specification

Overview

Semantic analysis validates that programs are meaningful according to STARK's language rules. This phase occurs after parsing and before code generation.

Analysis Phases

1. Symbol Table Construction

Build symbol tables for scopes and declarations.

Scope Rules

```
// Module scope
fn module_function() { }
const MODULE_CONST: Int32 = 42;

fn example() {
    // Function scope
    let local_var = 10;

    {
        // Block scope
        let block_var = 20;
        // block_var accessible here
        // local_var accessible here
    }
    // block_var not accessible here
    // local_var still accessible
}
```

Name Resolution Order

Name resolution distinguishes *lexical* scopes (inside function bodies) from *module* scopes.

Within a function body, an unqualified name is resolved lexically: 1. Current block scope 2. Enclosing block scopes (innermost to outermost) 3. Function parameters

If not found lexically, the name is resolved at module level per the module system rules (see 07-Modules-and-Packages.md): 4. Items declared in the current module 5. Items brought into scope by `use` 6. Built-in names (prelude)

Unqualified names never implicitly search parent modules or the crate root; those require explicit `super::` or `crate::` paths.

Shadowing Rules

```
let x = 10;       // x: Int32
{
    let x = "hi"; // Shadows outer x, x: String
    // Inner x takes precedence
}
// Outer x visible again
```

2. Type Checking

Variable Declarations

```
let x: Int32 = 42;     // Type annotation matches literal
let y = 42;            // Type inferred as Int32
let z: String = 42;    // Error: type mismatch
```

Function Calls

```
fn add(a: Int32, b: Int32) -> Int32 { a + b }

let result = add(1, 2);    // OK: arguments match parameters
let error = add(1.0, 2);   // Error: Float64 cannot be Int32
let error2 = add(1);       // Error: wrong number of arguments
```

Assignment Compatibility

```
let mut x: Int32 = 10;
x = 20;                    // OK: same type
x = "hello";               // Error: type mismatch

let y: Int32 = 10;
y = 20;                    // Error: y is not mutable
```

3. Ownership and Borrowing Analysis

Move Semantics Validation

```
let s1 = String::from("hello");
let s2 = s1;              // s1 moved to s2
print(s1);                // Error: use of moved value

fn take_ownership(s: String) { }
let s3 = String::from("world");
take_ownership(s3);       // s3 moved into function
print(s3);                // Error: use of moved value
```

Borrow Checking

```
let mut s = String::from("hello");
let r1 = &s;              // Immutable borrow
let r2 = &s;              // OK: multiple immutable borrows
let r3 = &mut s;          // Error: cannot borrow mutably while
immutably borrowed

let mut s2 = String::from("world");
let r4 = &mut s2;         // Mutable borrow
let r5 = &s2;             // Error: cannot borrow immutably while
mutably borrowed
let r6 = &mut s2;         // Error: cannot have multiple mutable
borrows
```

Lifetime Validation

Returned references must obey the Core v1 reference rules defined in 03-Type-System.md (“References and Lifetimes”): a returned reference must derive from a reference parameter on every path, and callers treat it as having the shortest lifetime among the reference arguments it may derive from.

```
fn invalid_return() -> &Int32 {
    let x = 42;
    &x                    // Error: returning reference to local
variable
}

fn valid_return(x: &Int32) -> &Int32 {
    x                     // OK: returning input reference
}
```

4. Control Flow Analysis

Unreachable Code Detection

```
fn example() -> Int32 {
    return 42;
    let x = 10;           // Warning: unreachable code
}
```

Return Path Analysis

```
fn missing_return() -> Int32 {
    let x = 42;
    // Error: not all paths return a value
}

fn conditional_return(flag: Bool) -> Int32 {
    if flag {
        return 1;
    }
```

```
    // Error: missing return in else branch
}

fn valid_return(flag: Bool) -> Int32 {
    if flag {
        1
    } else {
        2
    }                       // OK: both branches return
}
```

Break/Continue Validation

```
fn invalid_break() {
    break;                  // Error: break outside of loop
}

fn valid_break() {
    loop {
        break;              // OK: break inside loop
    }
}
```

5. Pattern Matching Analysis

Exhaustiveness Checking

```
enum Color { Red, Green, Blue }

fn check_color(c: Color) -> String {
    match c {
        Color::Red => "red",
        Color::Green => "green"
        // Error: non-exhaustive pattern (missing Blue)
    }
}

fn valid_match(c: Color) -> String {
    match c {
        Color::Red => "red",
        Color::Green => "green",
        Color::Blue => "blue"     // OK: exhaustive
    }
}
```

Rules: - A `match` over an enum type is exhaustive if every variant is covered, or a wildcard (_) arm exists. - Tuple patterns are exhaustive if each element position is exhaustive for its type. - Literal patterns are exhaustive only for finite domains (e.g., `Bool` with `true` and `false`). - If a match is not exhaustive, it is a compile-time error.

Pattern Type Checking

```
let x: Int32 = 42;
match x {
    "hello" => "string",          // Error: pattern type mismatch
    42 => "number",               // OK
    _ => "other"
}
```

6. Mutability Analysis

Mutable Access Validation

Assignment to an initialized immutable variable is an error. (Exception: the single initializing assignment of a deferred-initialization `let` — see Initialization Analysis.)

```
let x = 42;
x = 43;                   // Error: x is not mutable

let mut y = 42;
y = 43;                   // OK: y is mutable

struct Point { x: Int32, y: Int32 }
let p = Point { x: 1, y: 2 };
p.x = 10;                 // Error: p is not mutable

let mut p2 = Point { x: 1, y: 2 };
p2.x = 10;                // OK: p2 is mutable
```

7. Initialization Analysis

Use Before Initialization

```
let x: Int32;
print(x.fmt());           // Error: use of uninitialized variable

let y: Int32 = if true { 42 } else { 24 };
print(y.fmt());           // OK: y is initialized in all branches

let z: Int32;
if condition {
    z = 42;
}
print(z.fmt());           // Error: z might not be initialized
```

Rules: - `let name: Type;` declares a variable without initializing it. - A variable must be definitely assigned before any read. - All control-flow paths must assign before use. - **Deferred initialization of immutable variables**: an *uninitialized*

immutable `let` may be assigned exactly once on each control-flow path; this first assignment is initialization, not mutation. Any assignment to an already-initialized immutable variable is an error (see Mutability Analysis).

Double Initialization

```
let mut x: Int32 = 42;
x = 43;                   // OK: reassignment
let x = 44;               // Error: redeclaration in same scope
```

8. Array Bounds Analysis

Static Bounds Checking

```
let arr: [Int32; 3] = [1, 2, 3];
let x = arr[2];           // OK: index in bounds
let y = arr[5];           // Error: index out of bounds (if
determinable)
```

Dynamic Bounds Checking

```
let arr: [Int32; 3] = [1, 2, 3];
let idx = get_index();
let x = arr[idx];         // Runtime bounds check required
```

9. Error Propagation Analysis

Question Mark Operator

```
fn might_fail() -> Result<Int32, String> { Ok(42) }

fn caller1() -> Result<Int32, String> {
    let x = might_fail()?;    // OK: compatible error types
    Ok(x * 2)
}

fn caller2() -> Int32 {
    let x = might_fail()?;    // Error: function doesn't return
Result
    x * 2
}
```

Rules: - `expr?` propagates `Err` or `None` to the nearest enclosing function returning `Result<_, E>` or `Option<_>`. - The enclosing function return type must be compatible with the propagated type.

Runtime Error Semantics (Core v1)

- A runtime error (e.g., integer overflow, division by zero, out-of-bounds indexing, failing `as` cast) MUST terminate the current program execution.
- `panic(...)` is a runtime error that terminates the program after emitting the provided message.
- Termination is an **abort**: the program stops immediately with a non-zero exit status. Destructors (`Drop`) are NOT run for live values, and no unwinding or recovery mechanism exists in Core v1. (Catchable panics and unwind-with-destructors are possible future extensions.)

10. Trait Constraint Checking

Trait Bounds Validation

```
trait Display {
    fn fmt(&self) -> String;
}

fn print_it<T: Display>(item: T) {
    print(item.fmt());        // OK: T implements Display
}

struct Point { x: Int32, y: Int32 }

print_it(Point { x: 1, y: 2 });  // Error: Point doesn't implement
Display
```

Error Reporting

Error Message Format

```
Error: [ERROR_CODE] [BRIEF_DESCRIPTION]
  --> file.stark:line:column
   |
line | source code line
   | ^^^^^ specific location
   |
   = help: detailed explanation
   = note: additional information
```

Error Categories

Type Errors (E0001-E0099)

- E0001: Type mismatch
- E0002: Unknown type
- E0003: Type annotation required
- E0004: Cannot infer type

- E0005: Wrong number of arguments
- E0006: `?` operator in a function that does not return `Result` or `Option`
- E0007: Index out of bounds (determinable at compile time)

Ownership Errors (E0100-E0199)

- E0100: Use of moved value
- E0101: Borrow check failed
- E0102: Lifetime violation
- E0103: Dangling reference

Name Resolution Errors (E0200-E0299)

- E0200: Undefined variable
- E0201: Undefined function
- E0202: Undefined type
- E0203: Ambiguous name
- E0204: Duplicate definition in the same scope

Control Flow Errors (E0300-E0399)

- E0301: Missing return value
- E0302: Break outside loop
- E0303: Non-exhaustive match

(E0300 is intentionally unassigned: unreachable code is warning W0005, not an error — see "Unreachable Code Detection" above.)

Mutability and Initialization Errors (E0400-E0499)

- E0400: Assignment to immutable binding
- E0401: Use of possibly-uninitialized variable

Trait Errors (E0500-E0599)

- E0500: Trait not implemented

Warning Categories

Unused Items (W0001-W0099)

- W0001: Unused variable
- W0002: Unused function
- W0003: Unused import
- W0004: Dead code
- W0005: Unreachable code

Style Warnings (W0100-W0199)

- W0100: Non-snake_case variable
- W0101: Non-PascalCase type

- W0102: Missing documentation

Analysis Algorithm

Pass Order

1. **Declaration Pass**: Collect all top-level declarations
2. **Type Resolution Pass**: Resolve all type expressions
3. **Type Inference Pass**: Infer types for expressions
4. **Ownership Pass**: Check ownership and borrowing rules
5. **Control Flow Pass**: Analyze control flow and reachability
6. **Pattern Pass**: Check pattern exhaustiveness and types
7. **Constraint Pass**: Validate trait constraints and bounds

Dependency Resolution

- Forward references allowed for types and functions
- Circular dependencies detected and reported
- Initialization order determined for constants

Error Recovery

- Continue analysis after errors when possible
- Provide multiple related errors in single pass
- Suggest fixes when unambiguous
- Avoid cascading errors from single root cause

Implementation Considerations

Symbol Table Structure

```
struct SymbolTable {
    symbols: HashMap<String, Symbol>,
    parent: Option<&SymbolTable>,
    children: Vec<SymbolTable>
}

enum Symbol {
    Variable { ty: Type, mutable: bool, initialized: bool },
    Function { params: Vec<Type>, return_ty: Type },
    Type { definition: TypeDef },
    Constant { ty: Type, value: Value }
}
```

Type Checking Context

```
struct TypeContext {
    current_function: Option<FunctionId>,
```

```
    expected_return_type: Option<Type>,
    loop_depth: usize,
    borrowed_values: HashMap<ValueId, BorrowInfo>
}
```

Error Collection

```
struct ErrorReporter {
    errors: Vec<SemanticError>,
    warnings: Vec<SemanticWarning>,
    error_limit: usize
}
```

Conformance

A conforming Core v1 implementation MUST follow the requirements in this document. Any deviations or extensions MUST be explicitly documented by the implementation.

STARK Memory Model and Ownership Specification

Overview

STARK's memory model ensures memory safety without garbage collection through compile-time ownership tracking, similar to Rust but with some simplifications for the initial implementation.

Core Principles

1. Ownership

- Every value has exactly one owner
- When the owner goes out of scope, the value is dropped
- Ownership can be transferred (moved) but not duplicated

2. Borrowing

- Values can be borrowed without transferring ownership
- Immutable borrows allow reading
- Mutable borrows allow reading and writing
- Borrowing rules prevent data races at compile time

3. Lifetimes

- All references have a lifetime
- References cannot outlive the data they point to
- Core v1 has no lifetime annotations; the conservative rules in 03-Type-System.md ("References and Lifetimes") apply

Memory Layout

Stack Allocation

```
// Stack-allocated values
let x: Int32 = 42;              // x stored on stack
let y: [Int32; 4] = [1, 2, 3, 4];  // y stored on stack

struct Point { x: Float64, y: Float64 }
let p: Point = Point { x: 1.0, y: 2.0 };  // p stored on stack
```

Heap Allocation

```
// Heap-allocated values
let s: String = String::from("hello");   // String data on heap
let v: Vec<Int32> = Vec::new();          // Vec data on heap
let b: Box<Int32> = Box::new(42);        // Boxed value on heap
```

Reference Layout

```
// References are pointers (8 bytes on 64-bit)
let x: Int32 = 42;
let r: &Int32 = &x;         // r contains address of x

// Slice references contain pointer + length
let arr: [Int32; 5] = [1, 2, 3, 4, 5];
let slice: &[Int32] = &arr[1..4];  // pointer + length (16 bytes)
```

Ownership Rules

Rule 1: Single Ownership

```
let s1 = String::from("hello");
let s2 = s1;                // Ownership moved from s1 to s2
// println(s2.as_str());    // OK: s2 owns the string
// println(s1.as_str());    // Error: s1 no longer valid
```

Rule 2: Automatic Cleanup

```
{
    let s = String::from("hello");  // s owns the string
    // String is automatically freed when s goes out of scope
}
```

Rule 3: Function Parameters

```
fn take_ownership(s: String) {
    // s is owned by this function
    println(s.as_str());
    // s is dropped when function returns
}

fn main() {
    let s = String::from("hello");
    take_ownership(s);      // Ownership transferred to function
    // println(s.as_str()); // Error: s no longer valid
}
```

Move Semantics

What Gets Moved

Types are moved if they: 1. Don't implement the `Copy` trait 2. Contain non-Copy fields

```
// Types that are moved by default
String, Vec<T>, Box<T>, custom structs without Copy

// Types that are copied by default (implement Copy)
Int32, Float64, Bool, Char, &T, [T; N] where T: Copy
```

Move in Assignments

```
let s1 = String::from("hello");
let s2 = s1;                // Move
let s3 = s2.clone();        // Explicit copy (if Clone
implemented)
```

Move in Function Calls

```
fn process(s: String) { println(s.as_str()); }

let my_string = String::from("hello");
process(my_string);         // my_string moved into function
// my_string no longer accessible
```

Move in Returns

```
fn create_string() -> String {
    let s = String::from("hello");
    s                       // Ownership moved to caller
}
```

Partial Moves, Reinitialization, and Indexed Places

The normative rules are in 03-Type-System.md (“Copy and Drop”):

```
struct Person { name: String, age: Int32 }

let p = Person { name: String::from("Alice"), age: 30 };
let n = p.name;             // Partial move (Person does not
implement Drop)
let a = p.age;              // OK: remaining Copy field still
readable
// let q = p;               // Error: p is partially moved

let mut s = String::from("one");
let t = s;                  // s moved out
s = String::from("two");    // OK: reinitialization revalidates s

let v: Vec<String> = make();
// let x = v[0];            // Error: cannot move out of indexed
place
let x = &v[0];              // OK: borrow instead (or use
Vec::remove/pop)
```

Borrowing System

Immutable Borrowing

```
fn read_string(s: &String) -> UInt64 {
    s.len()                 // Can read but not modify
}

let my_string = String::from("hello");
let length = read_string(&my_string);  // Borrow my_string
println(my_string.as_str()); // my_string still accessible
```

Mutable Borrowing

```
fn modify_string(s: &mut String) {
    s.push('!');            // Can read and modify
}

let mut my_string = String::from("hello");
```

```
modify_string(&mut my_string);        // Mutable borrow
println(my_string.as_str()); // Prints "hello!"
```

Borrowing Rules

1. **Either** one mutable borrow **OR** any number of immutable borrows
2. References must always be valid

```
let mut s = String::from("hello");

// Multiple immutable borrows - OK
let r1 = &s;
let r2 = &s;

// Mutable and immutable borrow - Error
let r3 = &s;
let r4 = &mut s;            // Error: cannot borrow mutably while
immutably borrowed

// Multiple mutable borrows - Error
let r5 = &mut s;
let r6 = &mut s;            // Error: cannot borrow mutably twice
```

Lifetime System

Lifetime Basics

Core v1 applies the normative reference rules from 03-Type-System.md: returned references must derive from reference parameters; callers treat a returned reference as having the *shortest* lifetime among the reference arguments it may derive from; borrows bound to variables are lexically scoped (to end of block), while unbound temporary borrows end with their statement. User struct/enum declarations may not declare reference fields, but generic types instantiated with references (e.g. `Option<&T>`) are permitted as *borrow-carrying values* subject to the same rules as references — see “Borrow-Carrying Types” in 03-Type-System.md.

```
fn longest(x: &str, y: &str) -> &str {
    if x.len() > y.len() {
        x
    } else {
        y
    }
}
// The returned reference is treated as valid only while both x's
and y's
// referents are valid (shortest-input-lifetime rule).
```

Lifetime Annotations (Future Feature)

```
fn longest<'a>(x: &'a str, y: &'a str) -> &'a str {
    if x.len() > y.len() { x } else { y }
}
```

Dangling Reference Prevention

```
fn invalid() -> &Int32 {
    let x = 42;
    &x                      // Error: returning reference to local
variable
}

fn valid(x: &Int32) -> &Int32 {
    x                       // OK: returning input reference
}
```

Memory Management Strategies

Stack vs Heap Decision

```
// Stack allocated (small, known size)
let point = Point { x: 1.0, y: 2.0 };
let array = [1, 2, 3, 4, 5];

// Heap allocated (dynamic size, large objects)
let string = String::from("hello");
let vector: Vec<Int32> = Vec::new();
let boxed = Box::new(large_object);
```

Reference Counting (Rc/Arc)

```
// Single-threaded reference counting (future feature)
let data = Rc::new(v);
let data2 = data.clone();   // Increment reference count

// Multi-threaded reference counting (future feature)
let shared = Arc::new(v);
let shared2 = shared.clone();
```

Drop and Destructors

Automatic Drop

```
struct FileHandle {
    path: String,
    handle: Int32
```

```
}

impl Drop for FileHandle {
    fn drop(&mut self) {
        // Close file handle
        close_file(self.handle);
    }
}

{
    let file = FileHandle { path: String::from("test.txt"),
handle: 42 };
    // file.drop() called automatically when leaving scope
}
```

Drop Order

```
{
    let x = String::from("first");
    let y = String::from("second");
    let z = String::from("third");
    // Drop order: z, y, x (reverse declaration order)
}
```

Manual Drop

```
let s = String::from("hello");
drop(s);                    // Explicitly drop s
// println(s.as_str());     // Error: s has been dropped
```

Drop Soundness (Core v1)

Normative rules (see also “Copy and Drop” in 03-Type-System.md): - Destructors run **exactly once** per value — at scope exit, at explicit `drop(value)`, or when the owner is consumed. Implementations MUST track initialization state (drop flags) for values moved on only some paths. - `Drop::drop` cannot be called explicitly; only the `drop(value)` free function is allowed. - A type cannot implement both `Copy` and `Drop`. - Fields cannot be moved out of a value whose type implements `Drop`. - On a runtime error or `panic`, the program aborts immediately and destructors are NOT run (see 04-Semantic-Analysis.md, Runtime Error Semantics).

Copy vs Move Types

Copy Types

Implement `Copy` trait - assignment creates a copy, not a move:

```
// Built-in Copy types
Int8, Int16, Int32, Int64, UInt8, UInt16, UInt32, UInt64
Float32, Float64, Bool, Char, Unit
```

```
&T (immutable references), [T; N] where T: Copy

// Note: &mut T is NOT Copy (mutable borrows are exclusive and
move)

// Usage
let x: Int32 = 42;
let y = x;                  // x is copied, still accessible
// Both x and y valid here
```

Move Types

Do not implement `Copy` - assignment moves ownership:

```
// Built-in Move types
String, Vec<T>, Box<T>, HashMap<K, V>

// Custom types are Move by default
struct Person {
    name: String,
    age: Int32
}

let p1 = Person { name: String::from("Alice"), age: 30 };
let p2 = p1;                // p1 moved to p2
// println(p1.name.as_str());  // Error: p1 no longer valid
```

Smart Pointers

Box - Heap Allocation

```
let boxed_int = Box::new(42);
let large_array = Box::new([0; 1000]);  // Allocate large array on
heap
```

Rc - Reference Counting (Future)

```
let data = Rc::new(some_value);
let reference1 = data.clone();   // Increment reference count
let reference2 = data.clone();   // Increment reference count
// Data deallocated when all references dropped
```

RefCell - Interior Mutability (Future)

```
let data = RefCell::new(42);
{
    let mut borrowed = data.borrow_mut();  // Runtime borrow check
    *borrowed = 43;
```

```
}
// data.borrow() now yields 43
```

Memory Safety Guarantees

What STARK Prevents

1. **Null pointer dereferences**: No null pointers, use Option
2. **Buffer overflows**: Array bounds checking
3. **Use after free**: Ownership system prevents dangling pointers
4. **Double free**: Ownership ensures single deallocation
5. **Memory leaks**: Automatic cleanup through ownership
6. **Data races**: Borrowing rules prevent concurrent access

Runtime Checks

Some checks remain at runtime: - Array bounds checking (unless optimized away) - Integer overflow (traps in ALL build configurations; see Numeric Semantics in 03-Type-System.md) - RefCell borrow checking (future feature)

Performance Considerations

Zero-Cost Abstractions

- Ownership and borrowing have no runtime cost
- References are just pointers
- Move semantics avoid unnecessary copies

Optimization Opportunities

- Dead code elimination for unused values
- Lifetime optimization to reduce copies
- Stack allocation for values that do not escape

Memory Layout Optimization

- Struct field reordering to minimize padding
- Enum layout optimization for tagged unions
- Array and slice bounds check elimination

Implementation Notes

Compiler Phases

1. **Ownership Analysis**: Track value ownership through program
2. **Borrow Checking**: Validate borrowing rules

3. **Lifetime Inference**: Determine reference lifetimes
4. **Drop Insertion**: Insert drop calls at scope exits
5. **Memory Layout**: Determine stack vs heap allocation

Error Messages

```
Error: borrow of moved value
  --> example.stark:10:5
   |
 8 |     let s1 = String::from("hello");
   |         -- move occurs because `s1` has type `String`
 9 |     let s2 = s1;
   |              -- value moved here
10 |     println(s1.as_str());
   |             ^^ value borrowed here after move
```

Integration with Type System

- Ownership information included in type signatures
- Borrowing constraints encoded in function types
- Lifetime parameters for generic functions (future)

Future Extensions

Advanced Features

- Lifetime parameters and annotations
- Reference-carrying structs (requires lifetime annotations)
- Higher-ranked trait bounds
- Associated types with lifetime parameters
- Async/await with proper lifetime handling ## Conformance A conforming Core v1 implementation MUST follow the requirements in this document. Any deviations or extensions MUST be explicitly documented by the implementation.

STARK Standard Library Specification

Overview

The STARK standard library provides essential types, functions, and modules for core programming tasks. This specification defines the minimal standard library for the initial implementation.

Notation

The code blocks in this document are **API notation**, not compilable STARK source: function signatures are listed without bodies, and `impl` blocks mix signatures with prose comments. The *signatures and behaviors* are normative; the listing style is not. (The Core v1 grammar has no body-less function form outside `trait` blocks.)

Implementation-Provided Types

Several standard library types (`Box<T>`, `Vec<T>`, `String`, `HashMap<K, V>`) manage raw memory internally. Raw pointers and `unsafe` code are NOT part of the Core v1 language surface; these types are *implementation-provided*: their public APIs are normative, their internal representation is not expressible in Core v1 source and is implementation-defined. Struct bodies shown for these types in this document are illustrative only.

Iterator and View Types

The iterator types returned by the APIs below (`VecIter<T>`, `KeysIter<K>`, `ValuesIter<V>`, `Iter<K, V>`, `Iter<T>`, `CharsIter`, `SplitIter`, `MapIter<I, U>`, `FilterIter<I>`) are implementation-provided opaque structs. Each implements `Iterator` with the obvious `Item`:

Type	Produced by	`Item`
`VecIter<T>`	`Vec::iter`	`&T`
`KeysIter<K>`	`HashMap::keys`	`&K`
`ValuesIter<V>`	`HashMap::values`	`&V`
`Iter<K, V>`	`HashMap::iter`	`(&K, &V)`
`Iter<T>`	`HashSet::iter`	`&T`
`CharsIter`	`String::chars`, `str::chars`	`Char`
`SplitIter`	`String::split`	`&str`
`MapIter<I, U>`	`Iterator::map`	`U`
`FilterIter<I>`	`Iterator::filter`	`I::Item`

Iterators whose `Item` is a reference (and `CharsIter`/`SplitIter`, which borrow their source) are **borrow-carrying types** in the sense of 03-Type-System.md: they hold a live borrow of the collection they iterate and obey all reference rules (no storage in user structs, lexically scoped, cannot outlive their source).

Core Module Structure

```
std/
├── core/           // Core language items
├── collections/    // Data structures
├── io/            // Input/output operations
├── string/        // String manipulation
```

```
├── math/          // Mathematical operations
├── mem/           // Memory management utilities
├── error/         // Error handling types
└── prelude/       // Automatically imported items
```

Prelude Module

Automatically imported into every STARK program:

```
// Basic types (no import needed)
Int8, Int16, Int32, Int64, UInt8, UInt16, UInt32, UInt64
Float32, Float64, Bool, Char, String, str, Unit

// Essential traits
trait Copy
trait Clone
trait Drop
trait Eq
trait Ord
trait Hash
trait Default
trait Display

// Essential enums
enum Ordering {
    Less,
    Equal,
    Greater
}

enum Option<T> {
    Some(T),
    None
}

enum Result<T, E> {
    Ok(T),
    Err(E)
}

// Essential functions
fn print(value: &str)
fn println(value: &str)
fn panic(message: &str) -> !    // Never returns; see 03-Type-
System.md (Never Type)
```

Core Module (std::core)

Essential Trait Definitions

```
trait Clone {
    fn clone(&self) -> Self;
}

trait Eq {
    fn eq(&self, other: &Self) -> Bool;
}

trait Ord {
    fn cmp(&self, other: &Self) -> Ordering;
}

trait Hash {
    fn hash(&self) -> UInt64;
}

trait Default {
    fn default() -> Self;
}

trait Display {
    fn fmt(&self) -> String;
}

trait Drop {
    fn drop(&mut self);
}

// Copy is a marker trait (no methods); Copy: Clone.
// Soundness rules (all-fields-Copy, Copy/Drop exclusivity) are in
// 03-Type-System.md, "Copy and Drop".
trait Copy

// Num is a compiler-known marker trait implemented by exactly the
built-in
// numeric types; it enables arithmetic operators on generic
parameters
// (see 03-Type-System.md, "Operators and Traits"). Not user-
implementable.
trait Num
```

Indexing Traits

The `[]` operator desugars to these traits:

```
trait Index<Idx> {
    type Output;
    fn index(&self, index: Idx) -> &Self::Output;
```

```
}

trait IndexMut<Idx> {
    fn index_mut(&mut self, index: Idx) -> &mut Self::Output;
}
```

Range Types

The range operators `..` and `..=` produce these types:

```
struct Range<T> {
    start: T,
    end: T
}

struct RangeInclusive<T> {
    start: T,
    end: T
}

// Ranges over integer types are iterable
impl Iterator for Range<Int32> {
    type Item = Int32;
    fn next(&mut self) -> Option<Int32>;
}
// ... similarly for the other integer types, and for
RangeInclusive
```

Memory Management

```
// Box - heap allocation (implementation-provided; internals
opaque)
struct Box<T> { /* implementation-defined */ }

impl<T> Box<T> {
    fn new(value: T) -> Box<T>;
    fn into_inner(self) -> T;
}

// Manual memory management
fn drop<T>(value: T)
fn size_of<T>() -> UInt64      // T not inferable: call as
size_of::<Int32>()
fn align_of<T>() -> UInt64     // T not inferable: call as
align_of::<Int32>()
```

Option Type

```
enum Option<T> {
    Some(T),
    None
}
```

```
impl<T> Option<T> {
    fn is_some(&self) -> Bool;
    fn is_none(&self) -> Bool;
    fn unwrap(self) -> T;
    fn unwrap_or(self, default: T) -> T;
    fn map<U>(self, f: fn(T) -> U) -> Option<U>;
    fn and_then<U>(self, f: fn(T) -> Option<U>) -> Option<U>;
}
```

Result Type

```
enum Result<T, E> {
    Ok(T),
    Err(E)
}

impl<T, E> Result<T, E> {
    fn is_ok(&self) -> Bool;
    fn is_err(&self) -> Bool;
    fn unwrap(self) -> T;
    fn unwrap_or(self, default: T) -> T;
    fn map<U>(self, f: fn(T) -> U) -> Result<U, E>;
    fn map_err<F>(self, f: fn(E) -> F) -> Result<T, F>;
    fn and_then<U>(self, f: fn(T) -> Result<U, E>) -> Result<U,
E>;
}
```

Collections Module (std::collections)

Vec - Dynamic Array

```
// Implementation-provided; internals opaque
struct Vec<T> { /* implementation-defined */ }

impl<T> Vec<T> {
    fn new() -> Vec<T>;
    fn with_capacity(capacity: UInt64) -> Vec<T>;
    fn push(&mut self, item: T);
    fn pop(&mut self) -> Option<T>;
    fn len(&self) -> UInt64;
    fn capacity(&self) -> UInt64;
    fn is_empty(&self) -> Bool;
    fn get(&self, index: UInt64) -> Option<&T>;
    fn get_mut(&mut self, index: UInt64) -> Option<&mut T>;
    fn insert(&mut self, index: UInt64, item: T);
    fn remove(&mut self, index: UInt64) -> T;
    fn clear(&mut self);
    fn append(&mut self, other: &mut Vec<T>);
    fn extend<I: Iterator<Item = T>>(&mut self, iter: I);
    fn iter(&self) -> VecIter<T>;
```

```
    fn as_slice(&self) -> &[T];
}

// Index access
impl<T> Index<UInt64> for Vec<T> {
    type Output = T;
    fn index(&self, index: UInt64) -> &T;
}

impl<T> IndexMut<UInt64> for Vec<T> {
    fn index_mut(&mut self, index: UInt64) -> &mut T;
}
```

HashMap<K, V> - Hash Table

```
// Implementation-provided; internals opaque
struct HashMap<K, V> { /* implementation-defined */ }

impl<K: Hash + Eq, V> HashMap<K, V> {
    fn new() -> HashMap<K, V>;
    fn with_capacity(capacity: UInt64) -> HashMap<K, V>;
    fn insert(&mut self, key: K, value: V) -> Option<V>;
    fn get(&self, key: &K) -> Option<&V>;
    fn get_mut(&mut self, key: &K) -> Option<&mut V>;
    fn remove(&mut self, key: &K) -> Option<V>;
    fn contains_key(&self, key: &K) -> Bool;
    fn len(&self) -> UInt64;
    fn is_empty(&self) -> Bool;
    fn clear(&mut self);
    fn keys(&self) -> KeysIter<K>;
    fn values(&self) -> ValuesIter<V>;
    fn iter(&self) -> Iter<K, V>;
}
```

HashSet - Hash Set

```
// Implementation-provided; internals opaque
struct HashSet<T> { /* implementation-defined */ }

impl<T: Hash + Eq> HashSet<T> {
    fn new() -> HashSet<T>;
    fn insert(&mut self, value: T) -> Bool;
    fn remove(&mut self, value: &T) -> Bool;
    fn contains(&self, value: &T) -> Bool;
    fn len(&self) -> UInt64;
    fn is_empty(&self) -> Bool;
    fn clear(&mut self);
    fn iter(&self) -> Iter<T>;
}
```

String Module (std::string)

String Type

```
// Implementation-provided; internals opaque (UTF-8 byte buffer)
struct String { /* implementation-defined */ }

impl String {
    fn new() -> String;               // Empty string
    fn with_capacity(capacity: UInt64) -> String;
    fn from(s: &str) -> String;       // Construct from a string
slice/literal
    fn len(&self) -> UInt64;
    fn is_empty(&self) -> Bool;
    fn push(&mut self, ch: Char);
    fn push_str(&mut self, s: &str);
    fn pop(&mut self) -> Option<Char>;
    fn clear(&mut self);
    fn chars(&self) -> CharsIter;
    fn bytes(&self) -> &[UInt8];
    fn as_str(&self) -> &str;
    fn into_bytes(self) -> Vec<UInt8>;
    fn substring(&self, start: UInt64, end: UInt64) -> &str;
    fn contains(&self, pattern: &str) -> Bool;
    fn starts_with(&self, pattern: &str) -> Bool;
    fn ends_with(&self, pattern: &str) -> Bool;
    fn find(&self, pattern: &str) -> Option<UInt64>;
    fn replace(&self, from: &str, to: &str) -> String;
    fn split(&self, delimiter: &str) -> SplitIter;
    fn trim(&self) -> &str;
    fn to_lowercase(&self) -> String;
    fn to_uppercase(&self) -> String;
}

// String literals (&str)
impl str {
    fn len(&self) -> UInt64;
    fn is_empty(&self) -> Bool;
    fn chars(&self) -> CharsIter;
    fn bytes(&self) -> &[UInt8];
    fn to_string(&self) -> String;
    // ... similar methods to String
}
```

Math Module (std::math)

Basic Operations

```
// Constants
const PI: Float64 = 3.141592653589793;
```

```
const E: Float64 = 2.718281828459045;

// Basic functions
fn abs<T: Num>(x: T) -> T
fn min<T: Ord>(a: T, b: T) -> T
fn max<T: Ord>(a: T, b: T) -> T
fn clamp<T: Ord>(value: T, min: T, max: T) -> T

// Floating point functions
fn sqrt(x: Float64) -> Float64
fn pow(base: Float64, exp: Float64) -> Float64
fn log(x: Float64) -> Float64
fn log10(x: Float64) -> Float64
fn exp(x: Float64) -> Float64

// Trigonometric functions
fn sin(x: Float64) -> Float64
fn cos(x: Float64) -> Float64
fn tan(x: Float64) -> Float64
fn asin(x: Float64) -> Float64
fn acos(x: Float64) -> Float64
fn atan(x: Float64) -> Float64
fn atan2(y: Float64, x: Float64) -> Float64

// Rounding functions
fn floor(x: Float64) -> Float64
fn ceil(x: Float64) -> Float64
fn round(x: Float64) -> Float64
fn trunc(x: Float64) -> Float64

// Random numbers (simple linear congruential generator)
struct Random {
    seed: UInt64
}

impl Random {
    fn new(seed: UInt64) -> Random;
    fn next_int(&mut self) -> UInt64;
    fn next_float(&mut self) -> Float64;
    fn range(&mut self, min: Int32, max: Int32) -> Int32;
}
```

IO Module (std::io)

Basic IO Operations

```
// Standard streams
fn print(text: &str)
fn println(text: &str)
fn eprint(text: &str)     // stderr
fn eprintln(text: &str)   // stderr
```

```
// Simple file operations
struct File { /* implementation-defined */ }

impl File {
    fn open(path: &str) -> Result<File, IOError>;
    fn create(path: &str) -> Result<File, IOError>;
    fn read_to_string(&mut self) -> Result<String, IOError>;
    fn write(&mut self, data: &[UInt8]) -> Result<UInt64,
IOError>;
    fn write_str(&mut self, text: &str) -> Result<UInt64,
IOError>;
    fn close(self) -> Result<Unit, IOError>;
}

// Error types
enum IOError {
    NotFound,
    PermissionDenied,
    AlreadyExists,
    InvalidInput,
    Other(String)
}

// Utility functions
fn read_file(path: &str) -> Result<String, IOError>
fn write_file(path: &str, content: &str) -> Result<Unit, IOError>
```

Error Module (std::error)

Error Trait

```
trait Error {
    fn message(&self) -> String;
}

// Standard error types
struct GenericError {
    message: String
}

impl Error for GenericError {
    fn message(&self) -> String {
        self.message.clone()
    }
}
```

Note: error *chaining* (a `source()` method returning a trait object) requires `dyn Trait` support, which is a future extension. Core v1 errors carry a message only; richer error types can embed context in their own fields.

Memory Module (std::mem)

Memory Utilities

```
// Memory operations
fn size_of<T>() -> UInt64      // Call with turbofish:
size_of::<Int32>()
fn align_of<T>() -> UInt64     // Call with turbofish:
align_of::<Int32>()
fn swap<T>(a: &mut T, b: &mut T)
fn replace<T>(dest: &mut T, src: T) -> T
fn take<T: Default>(dest: &mut T) -> T
```

Raw-pointer copy operations (`copy`, `copy_nonoverlapping`) require raw pointers and `unsafe`, which are not part of Core v1; they are deferred to a future extension.

Iterator Trait (std::iter)

Basic Iterator Interface

```
trait Iterator {
    type Item;

    fn next(&mut self) -> Option<Self::Item>;

    // Default implementations
    fn count(self) -> UInt64;
    fn collect<C: FromIterator<Self::Item>>(self) -> C;
    fn map<U>(self, f: fn(Self::Item) -> U) -> MapIter<Self, U>;
    fn filter(self, predicate: fn(&Self::Item) -> Bool) ->
FilterIter<Self>;
    fn fold<B>(self, init: B, f: fn(B, Self::Item) -> B) -> B;
    fn reduce(self, f: fn(Self::Item, Self::Item) -> Self::Item)
-> Option<Self::Item>;
    fn any(self, predicate: fn(Self::Item) -> Bool) -> Bool;
    fn all(self, predicate: fn(Self::Item) -> Bool) -> Bool;
    fn find(self, predicate: fn(&Self::Item) -> Bool) ->
Option<Self::Item>;
}

trait FromIterator<T> {
    fn from_iter<I: Iterator<Item = T>>(iter: I) -> Self;
}
```

Limitation (Core v1): the combinator parameters above are *non-capturing* function types (`fn(...)`). They accept named functions and function references but cannot capture local variables. Capturing closures are a future extension; until then, prefer explicit loops when state must be carried into the operation.

Conversion Traits

Basic Conversion

```
trait From<T> {
    fn from(value: T) -> Self;
}

trait Into<T> {
    fn into(self) -> T;
}

trait TryFrom<T> {
    type Error;
    fn try_from(value: T) -> Result<Self, Self::Error>;
}

trait TryInto<T> {
    type Error;
    fn try_into(self) -> Result<T, Self::Error>;
}

// String conversion
trait ToString {
    fn to_string(&self) -> String;
}

trait FromStr {
    type Error;
    fn from_str(s: &str) -> Result<Self, Self::Error>;
}
```

Essential Trait Implementations

Default Implementations

```
// Copy trait for basic types
impl Copy for Int8 { }
impl Copy for Int16 { }
impl Copy for Int32 { }
impl Copy for Int64 { }
impl Copy for UInt8 { }
impl Copy for UInt16 { }
impl Copy for UInt32 { }
impl Copy for UInt64 { }
impl Copy for Float32 { }
impl Copy for Float64 { }
impl Copy for Bool { }
impl Copy for Char { }
impl Copy for Unit { }
```

```
// Clone for all Copy types (blanket implementation)
impl<T: Copy> Clone for T {
    fn clone(&self) -> T { *self }
}

// Eq trait for basic types
impl Eq for Int32 {
    fn eq(&self, other: &Int32) -> Bool { *self == *other }
}
// ... similar for other types

// Ord trait for basic types
impl Ord for Int32 {
    fn cmp(&self, other: &Int32) -> Ordering {
        if *self < *other { Ordering::Less }
        else if *self > *other { Ordering::Greater }
        else { Ordering::Equal }
    }
}
```

Conformance Profiles

Two standard-library conformance profiles are defined. A conforming implementation MUST state which profile it implements.

Profile: `core-min` (MVP)

The minimum standard library for Core v1 conformance: - Prelude: primitive types, `Option`, `Result`, `Ordering`, essential traits (`Copy`, `Clone`, `Drop`, `Eq`, `Ord`, `Num`), `print`, `println`, `panic` - `String` and `str` (construction, `len`, `push`/`push_str`, `as_str`, `chars`) - `Vec<T>` (construction, `push`, `pop`, `len`, `get`/`get_mut`, indexing) - `Range`/`RangeInclusive` with integer `Iterator` impls (for `for` loops) - `Box<T>` - `drop`, `size_of`, `align_of`

Profile: `std-full`

Everything in this document: `core-min` plus `HashMap`, `HashSet`, the `Iterator` trait with combinators and all iterator/view types, the full `String` API, the math module, file IO, `std::mem` utilities, the conversion traits, and `Hash`/`Default`/`Display`/`Index`/`IndexMut`.

Items in *neither* profile (informative future work, not required for any Core v1 conformance claim): buffered IO, regular expressions, time/date, threads and concurrency primitives, `Rc`/`Arc`/`RefCell`.

Platform Considerations

Cross-platform Abstractions

- File path handling
- Directory operations
- Environment variables
- Process spawning (future)

Performance Notes

- Vec uses exponential growth strategy
- HashMap uses open addressing with Robin Hood hashing
- String operations are UTF-8 aware
- Iterator chains compile to efficient loops

Behavioral Requirements (Core v1)

- Indexing `Vec<T>` with `[]` MUST perform bounds checking and MUST trap on out-of-bounds access.
- `get`/`get_mut` MUST return `None` for out-of-bounds indices and MUST NOT trap.
- Slicing an array, slice, or `Vec<T>` with a range MUST trap if the range is out of bounds or inverted.
- `String::substring(start, end)` MUST validate UTF-8 boundaries and MUST trap on invalid boundaries or ranges.
- IO functions MUST return `Result` with a non-`Ok` variant on failure and MUST NOT silently ignore errors. ## Conformance A conforming Core v1 implementation MUST implement at least the `core-min` profile, MUST state which profile it implements, and MUST follow the requirements in this document for every item it provides. Any deviations or extensions MUST be explicitly documented by the implementation.

STARK Modules and Packages Specification

Overview

This document defines the module system, visibility rules, and package resolution for the STARK core language. It is normative for Core v1.

Package Layout

A package is a directory containing a `starkpkg.json` manifest at its root.

- The manifest's `entry` field defines the root source file.
- If `entry` is omitted, the default is `src/main.stark`.

All module paths are resolved relative to the package root unless otherwise noted.

Module Declarations

Modules are declared with the `mod` keyword.

```
mod math;

mod inline {
    pub fn add(a: Int32, b: Int32) -> Int32 { a + b }
}
```

File-Based Modules

A declaration `mod name;` loads one of the following files, in order: 1. `name.stark` 2. `name/mod.stark`

The loaded file defines the contents of the module `name`.

Module Paths

Paths use `::` separators.

- `crate` refers to the package root module.
- `self` refers to the current module.
- `super` refers to the parent module.

Examples:

```
use crate::utils::math::add;
use super::config;
```

Imports (`use`)

The `use` statement brings names into scope.

```
use crate::utils::math::add;
use crate::utils::{math, io};
use crate::utils::math as m;
use crate::utils::*;
```

Rules: - `use` affects name resolution in the current module only. - `pub use` re-exports the imported name from the current module. - Aliases with `as` are local to the current module.

Visibility

- Items are private to their defining module by default.
- `pub` makes an item visible to parent modules and external modules.
- `priv` explicitly marks an item as private (same as default).

Visibility applies to: - Functions, structs, enums, traits, impl blocks, consts, type aliases, and modules.

Name Resolution Order

Within a module, names are resolved in the following order: 1. Local (lexical) scope — block scopes and function parameters, as defined in 04-Semantic-Analysis.md 2. Items declared in the current module 3. Items brought into scope by `use` 4. Built-in names (prelude)

Unqualified names do not implicitly search parent or crate scopes. Items in parent modules or the crate root require explicit `super::` or `crate::` paths. (Note: *lexical* scopes inside a function body do nest — step 1 — but *module* scopes never nest implicitly.)

Manifest Schema (`starkpkg.json`)

The manifest is a JSON object with the following fields:

Field	Type	Required	Rules
`name`	string	yes	Package name: `[a-z][a-z0-9_-]*`, 1–64 chars. Used as the import root for dependents.
`version`	string	yes	Semantic version `MAJOR.MINOR.PATCH` (numeric components; optional `-prerelease` tag).
`entry`	string	no	Package-root-relative path to the root source file. Default `src/main.stark`. MUST exist and MUST be inside the package directory.
`dependencies`	object	no	

Field	Type	Required	Rules
			Map from package name to a *version constraint string* or a *dependency object*.

A dependency object has exactly one of: - `{ "version": "<constraint>" }` — resolved from a registry or cache, or - `{ "path": "<relative-path>" }` — a local package directory containing its own `starkpkg.json`.

Version constraint syntax: - `"1.2.3"` — exactly that version - `"^1.2.3"` — `>=1.2.3` and `<2.0.0` (compatible-with) - `">=1.2, <2.0"` — comma-separated comparator list (`>=`, `>`, `<=`, `<`, `=`), all of which must hold

Validation: unknown top-level fields are ignored (forward compatibility); a missing/invalid required field, a malformed constraint, a `path` escaping the workspace, or a dependency whose name violates the name rule is a manifest error and compilation MUST fail.

Example:

```
{
  "name": "my-app",
  "version": "0.1.0",
  "entry": "src/main.stark",
  "dependencies": {
    "tensor-lib": "^2.1.0",
    "utils": { "path": "../utils" }
  }
}
```

Packages and Dependencies

Dependencies are declared in `starkpkg.json` under `dependencies`.

Resolution order for external packages: 1. Local package source (the current package) 2. Direct dependencies from `starkpkg.json` 3. Standard library package `std`

Dependency modules are accessed via their package name:

```
use TensorLib::tensor::Tensor;
```

Dependency Version Resolution (Core v1)

- Version strings follow semantic versioning.
- If multiple versions satisfy a constraint, the highest version MUST be selected.
- If no version satisfies a constraint, compilation MUST fail with an error.
- The source of packages (registry, cache, or local path) is implementation-defined, but the chosen version MUST be reported in build output.

Standard Library

The standard library is available under the `std` package name:

```
use std::io;
use std::collections::Vec;
```

Cycles

- **Package dependency cycles** — direct (`A → A`) or transitive (`A → B → C → A`) — are a compile-time error.
- **Module declarations** (`mod`) form a tree rooted at the package entry file, so `mod` cycles cannot occur; declaring the same module twice is an error.
- **`use` imports** between modules of the same package MAY be mutually recursive (module A may `use` items from B and vice versa); this is not a cycle error.

Errors

- Importing an unknown module or item is a compile-time error.
- Accessing a private item outside its module is a compile-time error.
- Ambiguous imports are a compile-time error unless aliased. ## Conformance A conforming Core v1 implementation MUST follow the requirements in this document. Any deviations or extensions MUST be explicitly documented by the implementation.
